ICT. Ed. 535: Visual Programming

Course No.: ICT. Ed 535 | Nature of course: Theoretical + Practical

Level: M.Ed. | Credit Hours: 3

Semester: Third | Teaching Hour: 64 hours (32+32)

1. Course introduction

This course provides modern software development skills with a graphical user interface using C# with ASP.net. The course covers most of the C# language. Students will build window- and web-based forms, add controls, and set their properties.

2. Objective of the course

- Allows students to create user controls in Windows Forms applications using the Visual Programming Platform.
- The student will prove proficiency in Flow Control and Decision-Making using C# code syntax and language elements.
- Provide students with the knowledge and skills needed to validate user input in Windows Forms applications
- To enable students to link Windows Forms applications to various data sources using Microsoft ADO.NET
- Enhance students' ability to create ASP.NET Web Application projects using .NET.
- To enable student to work with new .NET technologies using Blazor and Xamarin

3. Course Outlines

Specific objectives	Course contents	Teaching hours (T+P)
• Describe the need for visual programming and how it differs from traditional text-based programming. • Explore the graphical user interface (GUI) and its importance in visual programming. • Explain about Rapid Application Development (RAD) tools and techniques • Understand the event-driven programming paradigm • Describe how to set up and configure the development environment and ide, including visual studio and vs code.	**Unit 1: Visual Programming and C#** 1.1. Overview of Visual Programming, The need of Visual Programming 1.2. Graphical User Interface and different types of Visual Programming Practices 1.3. Rapid Application Development (RAD) Tools and Techniques 1.4. Event driven programming paradigm and C# 1.5. .NET Features, programming model and Architecture 1.6. Environment and IDE Setup and Configuration: Visual Studio and VS Code **Practical Works:** • Implementing event-driven programming paradigm using C# and .NET features, such as delegates, events, and lambda expressions, to respond to user actions and system events • Setting up the development environment and IDE configuration, including Visual Studio and VS Code, for developing visual programming applications. This includes configuring settings, installing necessary components and extensions, and working with different project types.	**4T+2P**
• Describe how to convert one data	**Unit 2: C# Language Basics**	**2T + 2P**

प्रमाणित प्रतिलिपि
कीर्तिपुर
Faculty of Education
Office of the Dean
Balkhu, Kathmandu

type to another, including implicit and explicit conversions. • Describe and discuss about different types of loops, including for, while, and do-while loops, and understand their syntax and use cases. • Analyze the concept of multi-dimensional arrays and how they can be used to store and manipulate data. • Describe the concept of structures • Describe the concept of enumerations • Discuss the concept of regular expressions • Describe the concept of collections • Apply C# Language Basics in creating and writing programs	2.1. Data Types and namespaces 2.2. Type Conversion 2.3. Flow controls: Looping and iterations 2.4. Arrays 2.5. Structures, Enumerations 2.6. Strings, Regular Expressions and Collections **Practical Works:** • Creating C# programs that define different data types and use them in various ways • Using namespaces to organize code and prevent naming conflicts between classes • Writing programs that demonstrate different ways to convert data types. • Creating programs that use different types of loops to perform various tasks • Creating programs that use arrays to store and manipulate data • Using multi-dimensional arrays to represent complex data • Creating programs that define custom structures and enumerations to represent data in a more meaningful way • Creating programs that use strings and regular expressions to manipulate and validate user input • Using collections such as lists and dictionaries to store and manipulate data in a flexible and dynamic way.	
• Describe about classes and objects and how they are used to create software applications. • Analyze the concept of method overloading and overriding and how it allows you to define methods with the same name but different parameters. • Explain how to define and use abstract classes and methods • Analyze the concept of Inheritance, Interfaces, Encapsulation, Polymorphism, Abstraction and Visual Inheritance and how it allows you to create new classes based on existing ones. • Descrinbe the concept of throwing exceptions and how it allows you to handle errors • Understand the concept of parallel programming and its benefits. • Discuss the concept of concurrent collections • Demonstrate method overloading, method overriding, abstract classes,	**Unit 3: Object Oriented Concepts in C#** 3.1. OOP Structure, Class and Object 3.2. Method Overloading, Method Overriding 3.3. Abstract Classes and Abstract Methods 3.4. Inheritance, Interfaces, Encapsulation, Polymorphism, Abstraction and Visual Inheritance 3.5. Exceptional handling: System defined exceptions and custom exceptions, Throwing Exceptions 3.6. Parallel Programs, Concurrent Collections **Practical Works:** • Creating classes, instantiating objects, demonstrating method overloading, method overriding, abstract classes, and abstract methods. • Creating a class hierarchy that demonstrates inheritance and polymorphism, using interfaces to define a common set of behaviors, encapsulation, and visual inheritance is part of these works. • Exception handling practices include using system-defined exceptions and custom exception classes, and parallel programs and concurrent collections are part of the	**5T + 5P**

Faculty of Education
Office of the Dean
Balkhu, Kathmandu

and abstract methods. • Create a class hierarchy that demonstrates inheritance and polymorphism, using interfaces to define a common set of behaviors, encapsulation, and visual inheritance is part of these works. • Demonstrate the parallel program involves using multiple threads to perform a CPU-bound task, and concurrent collections are used to safely share data between threads in a multi-threaded program.	practical work. • Demonstration of the parallel program involves using multiple threads to perform a CPU-bound task, and concurrent collections are used to safely share data between threads in a multi-threaded program.	
• Describe about various data access technologies available, including ADO.NET, LINQ to SQL, and Entity Framework. • Learn about the different components of ADO.NET, including Connection, Command, DataReader, and DataSet. • Discuss how to connect to a database using ADO.NET and how to execute basic SQL commands. • Analyze the different types of .NET data providers and how to use them to connect to different databases. • Explain how to create and execute commands using the Command object. • Describne how to create and work with DataTable, DataRow, and DataColumn objects. • Analyze the concept of LINQ to SQL and how it can be used to perform database operations using LINQ queries. • Discuss the concept of WPF and how it can be used to create modern user interfaces. • Discuss the concept of data bindings and how it can be used to connect UI elements to data sources.	**Unit 4: ADO.NET and WPF (Windows Presentation Foundation)** 4.1. Data access Technologies 4.2. ADO.NET Architecture 4.3. Data access using ADO.NET 4.4. ADO.NET Connections: .NET Data providers, Connection classes, Strings, Pooling, Events 4.5. ADO.NET commands: Command object, creating, executing commands, 4.6. Batch queries and stored procedures 4.7. Datasets, DataTable, DataRow, DataColumn and updating database from a DataSet 4.8. LINQ to SQL and DataSet 4.9. WPF Controls, Layouts, Dialogs, Menus, Commands and Data bindings **<u>Practical Works:</u>** • Write a program that demonstrates how to interact with a database using different data access technologies, such as ADO.NET, Entity Framework. • Understand the architecture of ADO.NET, including the role of the Data Provider, Connection, Command, DataReader, and DataSet objects. • Write a program that demonstrates how to use ADO.NET to interact with a database, including connecting to a database, executing queries, and retrieving data. • Understand the different .NET data providers available in ADO.NET and their respective connection classes, such as SqlConnection and OleDbConnection. Use connection strings to connect to a database and handle connection pooling and events. • Understand the Command object in ADO.NET and how to create and execute commands, including batch queries and stored procedures. • Use Datasets, DataTables, DataRows, and DataColumns to store and manipulate data	**4T + 6P**

Faculty of Education
Office of the Dean
Balkhu, Kathmandu

	retrieved from a database. Use DataAdapters to update the database from a DataSet. • Use LINQ to SQL to retrieve and manipulate data from a database using LINQ queries. Compare and contrast with using Datasets and DataAdapters. • Create a Windows Presentation Foundation (WPF) application that uses different types of controls, such as buttons, text boxes, and data grids. Use layouts to arrange controls on the window and create dialogs and menus to interact with the user. Use commands and data bindings to respond to user input and update the user interface.	
• Describe how to create web applications using ASP.NET MVC and its features, including controllers, views, and models. • Describe how to create RESTful APIs using ASP.NET Web API and its features, including routing, controllers, and models. • Describe how to create dynamic web pages using Razor templates, including layouts, views, and partial views. • Explain how to perform CRUD (Create, Read, Update, Delete) operations on data using Razor Pages. • Desmonstrate skills to implement authentication and authorization in ASP.NET Core using different authentication schemes and authorization policies. • Demonstrate skills to create and use middleware in ASP.NET Core to modify the request and response pipeline. • Analyse and apply the different options for deploying ASP.NET Core applications, including Azure, Docker, and AWS.	**Unit 5: ASP.NET and ASP.NET Core** 5.1. C# and ASP.NET with ASP.NET Core 5.2. ASP.NET architecture 5.3. Web applications using ASP.NET and ASP.NET MVC 5.4. RESTful APIs with ASP.NET Web API 5.5. ASP.NET Core Razor templates 5.6. Razor pages and CRUD Operation 5.7. Authentication and Authorization 5.8. ASP.NET Core middleware 5.9. Deployment (Azure, Docker, AWS) **Practical Works:** • Writing C# code for web development using ASP.NET and ASP.NET Core • Understanding the architecture of ASP.NET, including the HTTP pipeline and request/response processing • Building web applications using ASP.NET and ASP.NET MVC, with models, views, and controllers • Creating RESTful APIs using ASP.NET Web API, with routes, actions, and authentication/authorization • Generating dynamic HTML content in ASP.NET Core with Razor templates and syntax • Using Razor pages for CRUD operations and user input/output, with model binding and dynamic content generation • Implementing authentication and authorization with standard schemes and role/attribute-based authorization • Understanding the role of middleware in the ASP.NET Core pipeline, with the ability to create custom middleware for specific functionality • Deploying ASP.NET and ASP.NET Core applications to cloud platforms like Azure, Docker, and AWS, with configuration and	**5T + 5P**

Faculty of Education
Office of the Dean
Balkhu, Kathmandu

	automation using deployment tools like Visual Studio.	
• Describe about the different types of Blazor, including server-side and client-side Blazor. • Analyze the Blazor component model and how to create UI and components. • Describe the concept of validation in a web application. • Describe the concept of Razor Class Libraries. • Demonstrate skills to create reusable components and libraries using Razor.	**Unit 6: Web Applications with Blazor** 6.1. Blazor overview 6.2. Building a web app with Blazor: UI, Components, Data binding, data sharing, events and event handlings 6.3. Blazor pages, layouts, routing, navigation 6.4. Reusable Blazor components using Layouts 6.5. DOM events with Blazor event handlers 6.6. Server-side and client-side validation to the form 6.7. Application interactivity with lifecycle events 6.8. Razor class library concept and creation. **Practical Works:** • Build a web application using Blazor, including designing the user interface, creating and using components, implementing data binding to display and manipulate data, sharing data between components, and handling events. • Create Blazor pages and layouts to organize the application's content and implement routing and navigation between pages. • Create reusable Blazor components using layouts and demonstrate how they can be used across multiple pages and applications. • Handle DOM events in Blazor using event handlers, such as onclick or onchange, and perform actions in response to those events. • Implement validation in a Blazor form to ensure that data entered by the user meets certain criteria. Use both server-side and client-side validation to provide a better user experience. • Use lifecycle events, such as OnInitialized or OnAfterRender, to add interactivity to a Blazor application, such as updating the UI in response to changes in data. • Understand the concept of a Razor class library in Blazor and create a library that can be reused across multiple Blazor applications.	**5T + 5P**
• Describe the concept of cross-platform development and its importance in software development. • Explain about the basic XAML elements, including layouts, controls, text, data bindings, styles, and graphics. • Explain how to use XAML in	**Unit 7: Cross-Platform Development with Xamarin** 7.1. Overview of Xamarin and Cross-Platform Development 7.2. MVC and MVVM design patterns for Cross-Platform Development 7.3. XAML Basics: Layouts, Controls, Text, Data bindings, Styles and Graphics 7.4. XAML and Xamarin	**7T + 7P**

Faculty of Education
Office of the Dean
Balkhu, Kathmandu

Xamarin development and how to define user interfaces using XAML. • Describe when to use Xamarin and when to use Xamarin.Forms for cross-platform development. • Analyze the different types of Xamarin form controls, including layouts, pages, views, and cells. • Discuss User Interface (Styles, Navigations, Maps, ListView) and Themes (Light, Dark, and Custom Build): • Describe the basics of working with databases in Xamarin development. • Explain how to use SQLite.NET to create, read, update, and delete records in a SQLite database. • Describe how to develop Android applications using Xamarin. • Explain how to publish and distribute Xamarin applications to app stores and other distribution channels.	7.5. Xamarin vs Xamarin.Forms 7.6. Xamarin Form Controls: Layout, Page, Views, Cells 7.7. Navigation: Patterns, Modal Views, Drill-downs, Tabbed views, Master-Detail views. 7.8. User Interface (Styles, Navigations, Maps, ListView) and Themes (Light, Dark and Custom build) 7.9. Working with SQLite.NET database 7.10. Android Development with Xamarin: Creating views, Calling Services, OAuth 2. 7.11. Deploying application **Practical Works:** • Install Xamarin on your development machine and create a "Hello World" application to test your installation. • Implement an application using the Model-View-Controller (MVC) and Model-View-ViewModel (MVVM) design patterns. • Create a XAML file that defines a simple user interface with layouts, controls, and text, apply styles and graphics to your user interface to enhance its appearance. • Create a Xamarin Forms project and use XAML to define the user interface, Use XAML to create custom controls for your Xamarin Forms application. • Create a sample application using both technologies and compare the development experience and end result. • Use the layout control to create a responsive user interface for your Xamarin Forms application. • Implement a modal view in your Xamarin Forms application. • Use the master-detail view pattern to create a navigation menu for your application. • User Interface (Styles, Navigations, Maps, ListView) and Themes (Light, Dark and Custom build): • Implement a custom theme in your Xamarin Forms application, use maps and location services to enhance the functionality of your application. • Create a list view that displays data in a user-friendly way. • Integrate SQLite.NET into your Xamarin Forms application. • Use SQLite.NET to create, read, update, and delete data from a local database. • Create a custom view and call a web service in your Android Xamarin Forms application. • Implement OAuth 2 authentication in your	

Faculty of Education
Office of the Dean
Balkhu, Kathmandu

	Android Xamarin Forms application. • Create a package of your Xamarin Forms application for the Android or iOS app store. • Deploy your application to a remote server using Microsoft Azure.	

4. Instructional Techniques

The techniques taught for this course are divided into two groups. The first group covers general teaching techniques that apply to most lessons. The second group covers specific teaching techniques applied to specific units.

4.1. General techniques

- Provide reading material to familiarize students with the units.
- Lectures, Q&A, discussions, brainstorming, practice and buzz sessions.

4.2. Special Instructional techniques

Demonstration is an essential teaching technique for all units of this course in the teaching and learning process. In particular, illustration by real work will be a specific teaching technique in this course. Details of suggested teaching techniques are presented below.

Lab works: Students must create a fully functional dynamic website that reflects all of the core technologies studied in this course.

Note: Specific teaching techniques may or may not be required for each unit covered in the course outline.

5. Evaluation

Internal Assessment	Semester Examination	External Practical Exam/ Viva	Total Marks
40 Points	40 Points	20 Points	100 Points

Note: *Students must pass an internal assessment, an external practice exam, and a separate semester exam.*

5.1. Internal evaluation (40 Points)

Internal assessment will be conducted by the subject teacher based on the following criteria:

a.	Class Attendance	5 points
b.	Learning activities and class performance	5 points
c.	First assignment (Written assignment)	10 points
d.	Second assignment (Case Study/Project work with presentation)	10 points
e.	Terminal Examination	10 Points
	Total	40 Points

5.2. Semester examination (20 Points)

The Examinations Department, Dean Office will conduct and organize the final examination at the end of semester.

a. Objective question (Multiple choice 10 questions x 1mark) 10 Points

प्रशासक परिषद्का कार्यालय
श्रीतिपुर

b.	Subjective answer questions (6 questions with 2 'OR' x 5 marks)	30 Points
	Total	40 Points

5.3. External practical Exam/ Viva (40 Points)

The practical part of the course will be assessed by an external examiner. Practical exam will be based on following criteria.

a.	Record Book	6 points
b.	Lab work Exam/ Case / Project	9 points
c.	VIVA	5 points
	Total	20 Points

6. Recommended books and Reference materials (including relevant published articles in nation and international journals)

Alessandro Del Sole (2022). Xamarin with Visual Studio: Launch your mobile development career by creating Android and iOS applications using .NET and C# (English Edition), BPB Publications.

David Pine (2022). Learning Blazor, O'Reilly.

Albahari, J., Albahari, B., & Drayton, P. (2012). C# 5.0 in a nutshell (5th Ed). Beijing ; Sebastopol: O'Reilly.

Esposito, D. (2014). Programming Microsoft ASP.NET MVC (Third edition). Sebastopol, California: O'Reilly Media, Inc.

Ian Griffiths (2012), Programming C# 5.0, O'Reilly Media, Inc.

Evjen, B., Hanselman, S., & Rader, D. (2010). Professional ASP.NET 4 in C# and VB. Indianapolis, IN: Wiley Pub.

Sharp, J. (2013). Microsoft Visual C# 2013 step by step.

Stellman, A., & Greene, J. (2013). Head first C# (Third edition). Beijing: O'Reilly.

***

Faculty of Education
Office of the Dean
Balkhu, Kathmandu